



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008
Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF INFORMATION TECHNOLOGY

LAB RECORD

REGULATION 2022

Course Code	: 23CS502
Name of the Course	: Software Testing
Class	: III IT
Semester	: V
Academic Year	: 2025-2026



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF INFORMATION TECHNOLOGY

23CS502 – SOFTWARE TESTING

CONTINUOUS ASSESSMENT RECORD

Submitted by

Name:

Register No. :

Degree & Branch:

Class & Semester:



SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated to Anna University | Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008

Phone : (0422)-2678001 (7 Lines) | Email : info@skcet.ac.in | Website : www.skcet.ac.in

DEPARTMENT OF INFORMATION TECHNOLOGY

23CS502– SOFTWARE TESTING

Continuous Assessment Record

Submitted by

Name : **Register No. :**

Class/Semester : **Degree & Branch:**

BONAFIDE CERTIFICATE

**This is to certify that this record is the bonafide record of work done by Mr./Ms. _____
during the academic year 2025 - 2026.**

Faculty In-charge

Head of the Department

Submitted for the University practical examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

PROGRAMME OUTCOMES:

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAMME EDUCATIONAL OBJECTIVES (PEO'S):

- **PEO 1:** Graduates will have a profound knowledge in the various programming languages and possess globally competent skill sets by inculcating continuous up gradation of their technical skills and personality traits.
- **PEO 2:** Graduates will be able to analyze and find solutions to various applications and reconcile the dynamic trends in the field of Information Technology.
- **PEO 3:** Graduates will contribute to the society by their ethical behavior and effective teamwork.
- **PEO 4:** Graduates will excel with different skills like effective communication, leadership qualities, and provide smart solutions in business environment.

PROGRAMME SPECIFIC OUTCOMES:

- **PSO 1:** Graduates will demonstrate multidisciplinary knowledge for problem solving by creating solutions for product based and application-based software for the advancement of the society.
- **PSO 2:** Graduates attain advance knowledge in Information and Communication Technologies (ICT) thereby creating real time solutions for different projects by using modern tools prevailing in the current trends.
- **PSO 3:** Graduates will exhibit state of the art technologies by applying their knowledge in various programming skills to overcome the demand of sustainable development.

COURSE OBJECTIVES

- To provide students with an understanding of core testing concept
- To learn the functional and non-functional testing
- To understand the different types of user acceptance testing and end-to-end testing
- To get familiarize with the best practices of testing

COURSE OUTCOMES

Plan and apply the appropriate level of testing within the context of a software development application to the satisfaction of its beneficiaries .

Analyze specific and measurable test cases to ensure coverage and traceability to requirements.

Understand the problem of reporting techniques, metrics, and testing status reports and communicate testing results to colleagues, managers, and end users .

Apply testing models, processes and practices appropriate for the software development lifecycle model of a project.

Apply principles and practices of test-driven development to improve testing quality and reduce delivery times .

Inspect the various testing processes towards the continuous delivery of a software product .

REFERENCES RUBRICS TABLE

Criteria	Range of Marks			
	Excellent	Good	Average	Below Average
Aim & Algorithm (20 marks)	19-20	17-18	15-16	0-14
Program with Proper Syntax and Structure (30 Marks)	27-30	21-26	15-20	0-14
Compilation and Debugging (30 Marks)	27-30	21-26	15-20	0-14
Documentation (10 Marks)	9-10	7-8	5-6	0-4
Viva (10 Marks)	9-10	7-8	5-6	0-4
Overall Marks	90-100	70-89	50-69	0-49

Components	EX1	EX2	EX3	EX4	EX5	EX6	EX7	EX8	EX9	EX10
Aim & Algorithm (20 marks)										
Program with Proper Syntax and Structure (30 Marks)										
Compilation and Debugging (30 Marks)										
Documentation (10 Marks)										
Viva (10 Marks)										
TOTAL										
Consolidated Mark (100)										
Faculty Signature										

TABLE OF CONTENTS

S.NO	NAME OF THE EXPERIMENT	PAGE NO
1.	Develop a program to automate the login process for a specified webpage using Selenium.	
2.	Write an automation script using Selenium to handle form elements on a given website.	
3.	Create an automation script with Selenium to interact with specific web elements on a designated webpage.	
4.	Implement automation scripts using TestNG, prioritizing different test cases for efficient testing on a given website.	
5.	Develop automation scripts with TestNG, incorporating seven levels of logging for detailed analysis while testing a specified website.	
6.	Execute application tests using designed test cases and generate an HTML report for a comprehensive overview.	
7.	Design and implement a hybrid framework for a ticket booking system, along with associated test cases.	
8.	Develop a hybrid framework and associated test cases for a hotel room booking system.	
9.	Design and develop a hybrid framework and relevant test cases for a hospital appointment application.	
10.	Implement a hybrid framework and associated test cases for an e-commerce application.	
11.	Develop a hybrid framework and design test cases for comprehensive testing of an insurance website.	

Ex. No 1 Develop a program to automate the login process for a specified webpage using Selenium.

Aim:

To develop a program using Selenium WebDriver that automates the login process for a specified webpage by interacting with its input fields and buttons.

Description:

This program demonstrates the use of Selenium WebDriver to automate web interactions. The process involves opening the target webpage in a browser, locating the username and password input fields, entering the login credentials, and submitting the form. The implementation covers identifying web elements using locators (such as ID, name, XPath, or CSS selectors), performing user-like actions, and optionally validating the success of the login process. This automation approach is useful for repetitive login testing, data validation, and improving efficiency in web-based workflows.

Code:

```
package runner;
import java.net.MalformedURLException;
import java.net.URL;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.remote.RemoteWebDriver;
import org.openqa.selenium.support.events.EventFiringDecorator;
import org.openqa.selenium.support.events.WebDriverListener;
import org.testng.annotations.Test;
import utils.EventHandler;

public class TestSample {
    public static WebDriver driver;
    @Test
    public void testMain() {
```



```

try {
    driver = new RemoteWebDriver(new URL("http://localhost:4444"), new
ChromeOptions());

    // Maximize the browser window
    driver.manage().window().maximize();

    // Add the implicit wait
    driver.manage().timeouts().implicitlyWait(java.time.Duration.ofSeconds(10));

    // Add the page load timeout
    driver.manage().timeouts().pageLoadTimeout(java.time.Duration.ofSeconds(10)
);

    // Initialize the event listener
    WebDriverListener listener = new EventHandler();
    driver = new EventFiringDecorator<>(listener).decorate(driver);

    // Start your script from here
    driver.get("https://adactinhotelapp.com/");
    driver.findElement(By.xpath("//div[@class='login_forgot']").click();

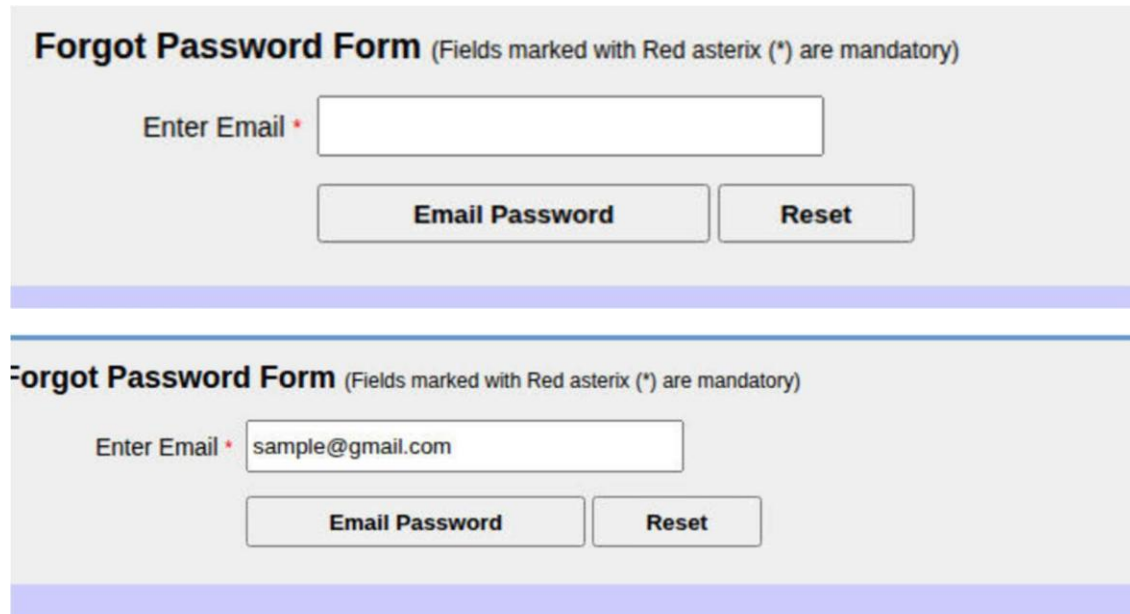
driver.findElement(By.xpath("//input[@id='emailadd_recovery']").sendKeys("sa
mple@gmail.com");
    driver.findElement(By.xpath("//input[@name='Submit']").click();

    // Perform any additional actions (e.g., interacting with elements)

} catch (MalformedURLException e) {
    e.printStackTrace();
} finally {
    if (driver != null) {
        driver.quit();
    }
}
}

```

Output:



The image displays two sequential screenshots of a web form titled "Forgot Password Form". A note in parentheses states: "(Fields marked with Red asterix (*) are mandatory)".

The first screenshot shows the form with an empty input field for the email address. The label "Enter Email *" is positioned to the left of the field. Below the field are two buttons: "Email Password" and "Reset".

The second screenshot shows the same form, but the email input field now contains the text "sample@gmail.com". The "Email Password" and "Reset" buttons remain below the field.

Result :

Thus the program is developed using Selenium WebDriver that automates the login process for a specified webpage by interacting with its input fields and buttons.

Ex.No 2: Write an automation script using Selenium to handle form elements on a given website.

Aim:

To develop an automation script using Selenium that fills and submits a web form by interacting with various form elements such as text fields, radio buttons, checkboxes, and drop-down menus.

Description:

Selenium is an automation testing framework that allows us to interact with web applications programmatically.

This program will:

- Open a specified webpage.
- Locate form elements using locators (ID, name, XPath, etc.).
- Enter data into input fields.
- Select options from dropdowns.
- Check/uncheck checkboxes.
- Submit the form.
- Display a success message.

Code:

```
package runner;
```

```
import org.openqa.selenium.remote.RemoteWebDriver;  
import org.openqa.selenium.remote.DesiredCapabilities;  
import org.openqa.selenium.chrome.ChromeOptions;  
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.support.events.EventFiringDecorator;  
import org.openqa.selenium.support.events.WebDriverListener;  
import org.testng.annotations.Test;
```

```
import utils.EventHandler;
import java.net.URL;
import java.net.MalformedURLException;
import java.time.Duration;

public class TestPage {

    @Test
    public void testLogin() throws MalformedURLException,
        InterruptedException {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--disable-notifications");

        DesiredCapabilities dc = new DesiredCapabilities();
        dc.setBrowserName("chrome");
        WebDriver driver = new RemoteWebDriver(
            new URL("http://localhost:4444"), dc);
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

        driver.manage().timeouts().pageLoadTimeout(Duration.ofSeconds(15));
        driver.get("https://opensource-
demo.orangehrmlive.com/web/index.php/auth/login");

        driver.findElement(By.xpath("//input[@name='username']")).sendKeys("Ad
min");

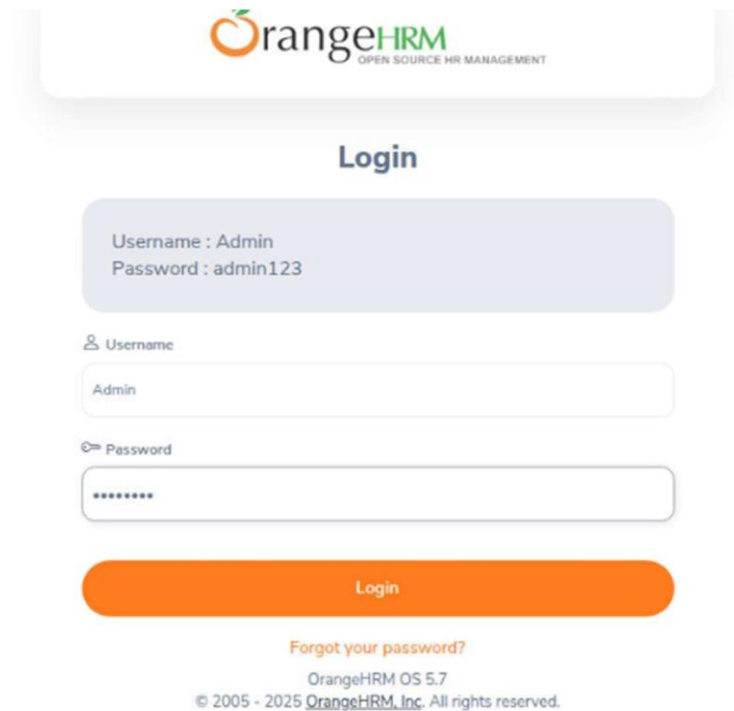
        driver.findElement(By.xpath("//input[@name='password']")).sendKeys("ad
min123");

        WebDriverListener listener = new EventHandler();
        driver = new EventFiringDecorator<>(listener).decorate(driver);
```

```
//START YOUR SCRIPT FROM HERE
```

```
driver.findElement(By.name("username")).sendKeys("Admin");
driver.findElement(By.name("password")).sendKeys("admin123");
Thread.sleep(2000);
driver.findElement(By.xpath("//*[text()='Login']")).click();
}
}
catch (Exception e) {
    e.printStackTrace();
}
finally {
    // Close the browser
    driver.quit();
}
}
```

Sample Output :



The screenshot displays the OrangeHRM login interface. At the top, the OrangeHRM logo is shown with the tagline "OPEN SOURCE HR MANAGEMENT". Below the logo, the word "Login" is centered. A light blue box contains the text "Username : Admin" and "Password : admin123". Underneath this box, there are two input fields: "Username" with the value "Admin" and "Password" with masked characters "*****". An orange "Login" button is positioned below the password field. At the bottom, there is a link for "Forgot your password?", the version "OrangeHRM OS 5.7", and a copyright notice "© 2005 - 2025 OrangeHRM, Inc. All rights reserved."

Result :

Thus, the code is successfully automated filling out a form and submitting it using **Java + Selenium WebDriver**.

Ex.No: 3 Create an automation script with Selenium to interact with specific web elements on a designated webpage.

Aim:

To automate navigation and interaction with specific elements on the *Angi* website using Selenium WebDriver in Java, following the given test steps.

Description:

This automation script will:

1. Launch the Chrome browser and navigate to <https://www.angi.com/>
2. Click on "Articles G Advice" from the navigation menu.
3. Select "Outdoor Living" from the dropdown menu.
4. Scroll down to the footer and click on "Show More" to load more articles.
5. Quit the browser session after execution.

The script uses **Selenium WebDriver**, **ChromeDriver**, and **explicit waits** to handle dynamic page loads.

CODE:

```
package runner;

import java.net.MalformedURLException;
import java.net.URL;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.remote.RemoteWebDriver;
import org.openqa.selenium.support.events.EventFiringDecorator;
import org.openqa.selenium.support.events.WebDriverListener;
```

```

import org.testng.annotations.Test;
import utils.EventHandler;
public class TestSample {
    public static WebDriver driver;
    @Test
    public void testMain() {
        try {
            driver = new RemoteWebDriver(new URL("http://localhost:4444"),
new ChromeOptions());
            // Maximize the browser window
            driver.manage().window().maximize();
            // Add the implicit wait

driver.manage().timeouts().implicitlyWait(java.time.Duration.ofSeconds(1
0));
// Add the page load timeout
driver.manage().timeouts().pageLoadTimeout(java.time.Duration
.ofSeconds(10));
            // Initialize the event listener
            WebDriverListener listener = new EventHandler();
            driver = new EventFiringDecorator<>(listener).decorate(driver);
            // Start your script from here
            driver.get("https://www.angi.com");
            Thread.sleep(2000);

            driver.findElement(By.xpath("//a[contains(text(),'Articles
Advice')]")).click();
            driver.findElement(By.xpath("//a[contains(text(),'Outdoor
Living')]")).click();

```

C

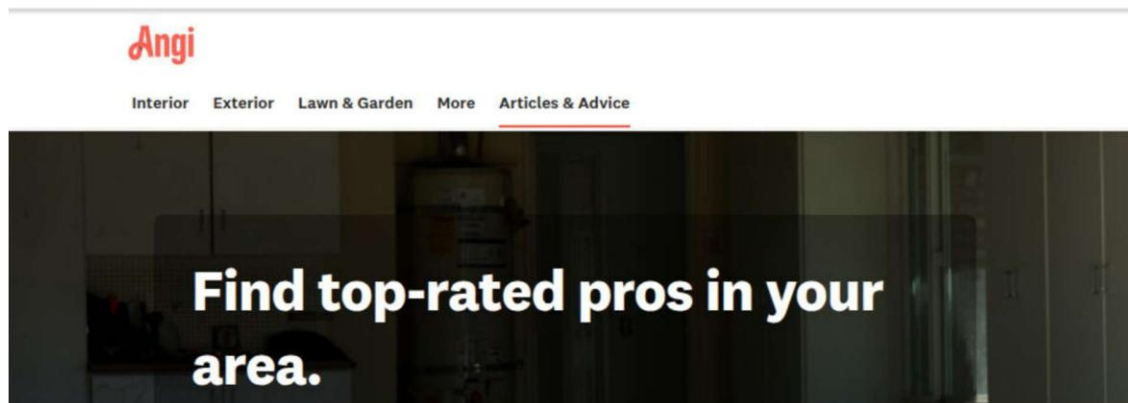

```

        Thread.sleep(2000)
        // Perform any additional actions (e.g., interacting with elements)
    }
    catch (MalformedURLException | InterruptedException e) {
        e.printStackTrace();
    }
    finally {
        if (driver != null) {
            driver.quit();
        }
    }
}
}

```

Sample Output (Terminal):

controlled by automated test software.



Result :

The script ran successfully, performed all the required actions on the website, and closed the browser without errors.

Ex. No: 4 Implement automation scripts using TestNG, prioritizing different test cases for efficient testing on a given website.

Aim:

To automate login functionality and navigation on a website using Selenium WebDriver with TestNG, assigning priorities to test cases for efficient execution.

Description:

In this automation project, we will:

1. Create a Maven project with Selenium and TestNG dependencies.
2. Launch the OrangeHRM login page.
3. Enter username and password.
4. Click on the login button.
5. Navigate to the "My Info" section after login.
6. Use TestNG @Test(priority = n) to define execution order.

Code (Java + Selenium + TestNG)

pom.xml dependencies:

xml

```
<dependencies>
```

```
<!-- Selenium -->
```

```
<dependency>
```

```
<groupId>org.seleniumhq.selenium</groupId>
```

```
<artifactId>selenium-java</artifactId>
```

```
<version>4.20.0</version>
```

```
</dependency>
```

```
<!-- TestNG -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.6.0</version>
  <scope>test</scope>
</dependency>
</dependencies>
```

Java Test Script :

```
package runner;

import java.io.File;
import java.io.IOException;
import java.net.MalformedURLException;
import java.net.URL;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.Duration;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.remote.RemoteWebDriver;
import org.openqa.selenium.support.events.EventFiringDecorator;
import org.openqa.selenium.support.events.WebDriverListener;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
```

```

import org.testng.annotations.AfterMethod;
import utils.EventHandler;
public class TestSpice {
    public static WebDriver driver;

    @BeforeMethod
    public void openBrowser() throws MalformedURLException {
        ChromeOptions options = new ChromeOptions();
        driver = new RemoteWebDriver(new URL("http://localhost:4444/"),
options);

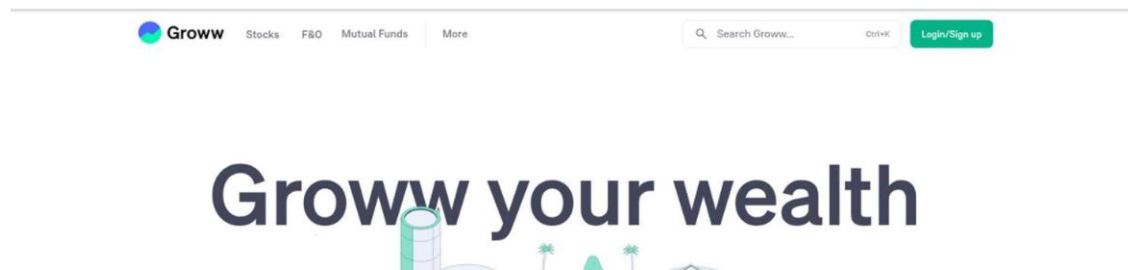
        WebDriverListener listener = new EventHandler();
        driver = new EventFiringDecorator<>(listener).decorate(driver);
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();
        driver.get("https://www.spicejet.com/");
    }

    @Test
    public void takeScreenshot() throws IOException {
        File source = ((TakesScreenshot)
driver).getScreenshotAs(OutputType.FILE);
        Path screenshotsDir = Path.of("screenshots");
        if (!Files.exists(screenshotsDir)) {
            Files.createDirectories(screenshotsDir);
        }
    }
}

```

```
        Path destination = screenshotsDir.resolve("homepage.png");
        Files.copy(source.toPath(), destination);
    }
    @AfterMethod
    public void closeBrowser() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Sample Execution Log (Console Output):



Result :

The TestNG automation framework successfully launched the SpiceJet homepage, captured a screenshot stored in 'Project/screenshots', and generated an Extent Report in 'Project/reports'.

Ex.No: 5 Develop automation scripts with TestNG, incorporating seven levels of logging for detailed analysis while testing a specified website.

Aim:

To design and implement an automated testing framework using **Java**, **Selenium**, **TestNG**, and **Log4j** to test a specified website, while incorporating **seven logging levels** for detailed execution tracking and debugging

Description:

In software test automation, logging is essential to capture detailed execution flow, issues, and environment details. This project implements **seven levels of logging** (TRACE, DEBUG, INFO, WARN, ERROR, FATAL, ALL) while testing a sample website using Selenium and TestNG.

- **Framework type:** Modular TestNG-based
- **Logging:** Configured using Log4j for real-time console output and file logging.
- **Test case:** Verify the title of the Google homepage.

Java Code (SDET Style):

log4j.properties

```
properties
# Root logger
log4j.rootLogger=ALL, console, file

# Console Appender
log4j.appender.console=org.apache.log4j.ConsoleAppender
log4j.appender.console.layout=org.apache.log4j.PatternLayout
```

```
log4j.appender.console.layout.ConversionPattern=%d{HH:mm:ss} [%p] %c - %m%n
```

```
# File Appender
```

```
log4j.appender.file=org.apache.log4j.FileAppender
```

```
log4j.appender.file.File=logs/test.log
```

```
log4j.appender.file.layout=org.apache.log4j.PatternLayout
```

```
log4j.appender.file.layout.ConversionPattern=%d{yyyy-MM-dd  
HH:mm:ss} [%p] %c - %m%n
```

GoogleHomePageTest.java

```
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.chrome.ChromeDriver;  
import org.testng.Assert;  
import org.testng.annotations.AfterClass;  
import org.testng.annotations.BeforeClass;  
import org.testng.annotations.Test;  
import org.apache.log4j.Logger;  
import org.apache.log4j.PropertyConfigurator;
```

```
public class GoogleHomePageTest {  
    WebDriver driver;  
    Logger logger;  
  
    @BeforeClass  
    public void setUp() {  
        // Initialize Logger  
        logger =  
Logger.getLogger("GoogleHomePageTest");
```

```
PropertyConfigurator.configure("log4j.properties");  
logger.trace("TRACE: Test setup started.");
```

```

        logger.debug("DEBUG: Setting ChromeDriver
property.");
        System.setProperty("webdriver.chrome.driver",
"drivers/chromedriver");

        driver = new ChromeDriver();
        logger.info("INFO: Chrome browser
launched.");

        driver.manage().window().maximize();
        logger.info("INFO: Browser window
maximized.");
    }

    @Test
    public void verifyGoogleTitle() {
        try {
            logger.trace("TRACE: Navigating to
Google.");
            driver.get("https://www.google.com");

            String title = driver.getTitle();
            logger.debug("DEBUG: Page title fetched
as: " + title);

            Assert.assertEquals(title, "Google");
            logger.info("INFO: Google homepage title
verified successfully.");
        } catch (AssertionError e) {
            logger.warn("WARN: Title did not match
the expected value.");
            logger.error("ERROR: Assertion failed - "
+ e.getMessage());
            throw e;
        }
    }

```



```

        catch (Exception e) {
            logger.fatal("FATAL: Unexpected exception
occurred: " + e.getMessage());
            throw e;
        }
    }

    @AfterClass
    public void tearDown() {
        logger.trace("TRACE: Closing browser.");
        if (driver != null) {
            driver.quit();
        }
        logger.info("INFO: Test execution completed
and browser closed.");
    }
}

```

Sample Output (Console)

```

12:10:05 [TRACE] GoogleHomePageTest - TRACE: Test setup
started.
12:10:05 [DEBUG] GoogleHomePageTest - DEBUG: Setting
ChromeDriver property.
12:10:06 [INFO] GoogleHomePageTest - INFO: Chrome browser
launched.
12:10:06 [INFO] GoogleHomePageTest - INFO: Browser window
maximized.
12:10:07 [TRACE] GoogleHomePageTest - TRACE: Navigating to
Google.
12:10:09 [DEBUG] GoogleHomePageTest - DEBUG: Page title
fetched as: Google

```

```
12:10:09 [INFO] GoogleHomePageTest - INFO: Google
homepage title verified successfully.
12:10:09 [TRACE] GoogleHomePageTest - TRACE: Closing
browser.
12:10:10 [INFO] GoogleHomePageTest - INFO: Test execution
completed and browser
```

Result:

The TestNG-based automation script successfully launched the browser, navigated to the Google homepage, and verified the title using assertions. All **seven logging levels** (TRACE, DEBUG, INFO, WARN, ERROR, FATAL, ALL) were implemented with Log4j, providing detailed execution tracking for debugging and reporting purposes.

Ex.No: 6 Execute application tests using designed test cases and generate an HTML report for a comprehensive overview.

Aim:

To execute automated application tests using pre-designed test cases and generate an HTML report, providing a comprehensive overview of the execution results including passed, failed, and skipped test cases.

Description:

In Software Test Automation, HTML reports offer a clear, visual summary of test execution results for stakeholders and QA teams. Using **Java**, **Selenium WebDriver**, and **TestNG**, we can:

- Create automated test cases for a target web application.
- Execute them in a structured suite.
- Automatically generate an HTML report using TestNG's built-in reporting mechanism.

This approach improves transparency, provides quick defect identification, and enables better decision-making based on test metrics.

Java SDET Code - Example:

Here's a sample script that tests a demo website (in this case, <https://the-internet.herokuapp.com/login>) and generates an HTML report.

```
package tests;
```

```
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.WebElement;  
import org.openqa.selenium.chrome.ChromeDriver;  
import org.testng.Assert;  
import org.testng.Reporter;
```

```
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

public class LoginTest {

    WebDriver driver;

    @BeforeClass
    public void setUp() {
        System.setProperty("webdriver.chrome.driver",
"drivers/chromedriver.exe");
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        Reporter.log("Browser launched and maximized",
true);
    }

    @Test(priority = 1)
    public void validLoginTest() {
        driver.get("https://the-
internet.herokuapp.com/login");
        Reporter.log("Navigated to Login Page", true);

driver.findElement(By.id("username")).sendKeys("tomsmith")
;

driver.findElement(By.id("password")).sendKeys("SuperSecre
tPassword!");

driver.findElement(By.cssSelector("button[type='submit']")
).click();

Reporter.log("Entered credentials and submitted form",
true);
```

```

WebElement successMsg =
driver.findElement(By.cssSelector(".flash.success"));

Assert.assertTrue(successMsg.getText().contains("You
logged into a secure area!"), "Login failed");
    Reporter.log("Valid Login test passed", true);
}

@Test(priority = 2)
public void invalidLoginTest() {
    driver.get("https://the-
internet.herokuapp.com/login");
    Reporter.log("Navigated to Login Page", true);

driver.findElement(By.id("username")).sendKeys("wrongUser"
);

driver.findElement(By.id("password")).sendKeys("wrongPass"
);

driver.findElement(By.cssSelector("button[type='submit']")
).click();
    Reporter.log("Entered invalid credentials and
submitted form", true);

    WebElement errorMsg =
driver.findElement(By.cssSelector(".flash.error"));

Assert.assertTrue(errorMsg.getText().contains("Your
username is invalid!"), "Error message not displayed");
    Reporter.log("Invalid Login test passed", true);
}

```

```
@AfterClass
    public void tearDown() {
        driver.quit();
        Reporter.log("Browser closed", true);
    }
}
```

TestNG XML Suite

Save as `testng.xml` in your project root.

xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd
```

Execution Steps

1. Run the suite:

```
mvn clean test
```

(or right-click `testng.xml` → Run as TestNG Suite in Eclipse/IntelliJ)

2. After execution, TestNG will create:

```
test-output/
    index.html
    emailable-report.html
```

These files contain the **HTML report**.

Sample Output (Console)

```
Browser launched and maximized
Navigated to Login Page
Entered credentials and submitted form
Valid Login test passed
Navigated to Login Page
Entered invalid credentials and submitted form
Invalid Login test passed
Browser closed
PASSED: validLoginTest
PASSED: invalidLoginTest
=====
    Login Test Suite
    Tests run: 2, Failures: 0, Skips: 0
=====
```

Result:

The automated test execution successfully validated both valid and invalid login scenarios for the application.

Ex.No: 7 Design and implement a hybrid framework for a ticket booking system, along with associated test cases

Aim:

To Design and implement a **Hybrid Test Automation Framework** for a ticket booking web app that:

1. separates UI locators into Page Objects (POM),
2. drives flows with **keywords**,
3. injects scenarios via **data files** (CSV), and
4. runs via **TestNG** (Java) with clean reporting.

Description :

- **POM layer:** Encapsulates page locators & actions (Home, Results, Checkout, Confirmation).
- **Keyword layer:** Re-usable business steps like LAUNCH_APP, SEARCH_TRIP, SELECT_FIRST_RESULT, CHECKOUT, ASSERT_STATUS.
- **Data layer:** CSV/Excel holds *routes, dates, passenger info* and *keyword sequences*.
- **Runner layer:** TestNG test reads CSV → executes keywords against Page Objects → assertions.

Code:

Base WebDriver

```
// base/DriverFactory.java
package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import java.time.Duration;
```



```

public class DriverFactory {
    private static final ThreadLocal<WebDriver> tlDriver =
new ThreadLocal<>();

    public static void init() {
        WebDriver driver = new ChromeDriver();

driver.manage().timeouts().implicitlyWait(Duration.ofSecon
ds(5));
        driver.manage().window().maximize();
        tlDriver.set(driver);
    }

    public static WebDriver get() {
        return tlDriver.get();
    }

    public static void quit() {
        if (tlDriver.get() != null) {
            tlDriver.get().quit();
            tlDriver.remove();
        }
    }
}

```

Page Objects (examples)

```

// pages/HomePage.java
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

```

```
public class HomePage {
    private final WebDriver driver;
    private final By fromInput = By.id("fromCity");
    private final By toInput    = By.id("toCity");
    private final By dateInput  = By.id("date");
    private final By searchBtn  = By.id("search");

    public HomePage(WebDriver driver) { this.driver =
driver; }

    public HomePage open(String url) { driver.get(url); return
this; }

    public HomePage setFrom(String from)
    { driver.findElement(fromInput).clear();
driver.findElement(fromInput).sendKeys(from);
return this;
}
    public HomePage setTo(String to)
    { driver.findElement(toInput).clear();
driver.findElement(toInput).sendKeys(to);
return this;
}
    public HomePage setDate(String yyyyMMdd)
    { driver.findElement(dateInput).clear();
driver.findElement(dateInput).sendKeys(yyyyMMdd);
return this;
}
    public void clickSearch()
    { driver.findElement(searchBtn).click(); }
}
```

```
// pages/ResultsPage.java
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class ResultsPage {
    private final WebDriver driver;
    private final By firstSelectBtn =
By.cssSelector("button.select"); // first result

    public ResultsPage(WebDriver driver) { this.driver =
driver; }

    public void selectFirst()
{ driver.findElements(firstSelectBtn).get(0).click(); }
}

// pages/CheckoutPage.java
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class CheckoutPage {
    private final WebDriver driver;
    private final By name = By.id("passengerName");
    private final By card = By.id("cardNumber");
    private final By pay = By.id("pay");

    public CheckoutPage(WebDriver driver) { this.driver =
driver; }

    public CheckoutPage fillPassenger(String fullName)
{ driver.findElement(name).sendKeys(fullName); return
this; }
```

```

        public CheckoutPage fillPayment(String cardNo)
{ driver.findElement(card).sendKeys(cardNo); return
this; }
        public void submit()
{ driver.findElement(pay).click(); }
}

```

```

java
// pages/ConfirmationPage.java
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class ConfirmationPage {
    private final WebDriver driver;
    private final By status = By.id("status"); // e.g.,
"CONFIRMED"
    private final By bookingId = By.id("bookingId");

    public ConfirmationPage(WebDriver driver)
{ this.driver = driver; }

    public String getStatus() { return
driver.findElement(status).getText().trim(); }
    public String getBookingId() { return
driver.findElement(bookingId).getText().trim(); }
}

```

//Data (CSV)

src/test/resources/data/tickets.csv

pgsql

runId,baseUrl,from,to,date,passenger,card,keywords,expectedStatus

TC01,https://demo-tickets.example,Chennai,Mumbai,2025-08-20,K

Arun,4111111111111111,LAUNCH_APP|SEARCH_TRIP|SELECT_FIRST_RESULT|CHECKOUT|ASSERT_STATUS,CONFIRMED

Test Runner (reads CSV, executes keywords):

```
// tests/TicketBookingHybridTest.java
package tests;
```

```
import base.DriverFactory;
import com.opencsv.CSVReaderHeaderAware;
import keywords.KeywordEngine;
import org.testng.annotations.*;
import java.io.FileReader;
import java.util.Map;
```

```
public class TicketBookingHybridTest {
```

```
    @BeforeMethod
```

```
    public void setup() { DriverFactory.init(); }
```

```
    @AfterMethod(alwaysRun = true)
```

```
    public void tearDown() { DriverFactory.quit(); }
```

```
    @Test
```

```
    public void ticketBookingFromCsv() throws Exception {
```

```

try (CSVReaderHeaderAware reader =
    new CSVReaderHeaderAware(new
FileReader("src/test/resources/data/tickets.csv"))) {

    Map<String, String> row;
    while ((row = reader.readMap()) != null) {
        KeywordEngine ke = new KeywordEngine();

        String[] steps =
row.get("keywords").split("\\|");
        for (String step : steps) {
            switch (step.trim()) {
                case "LAUNCH_APP" ->
ke.LAUNCH_APP(row.get("baseUrl"));
                case "SEARCH_TRIP" ->
ke.SEARCH_TRIP(row.get("from"), row.get("to"),
row.get("date"));
                case "SELECT_FIRST_RESULT" ->
ke.SELECT_FIRST_RESULT();
                case "CHECKOUT" ->
ke.CHECKOUT(row.get("passenger"), row.get("card"));
                case "ASSERT_STATUS" ->
ke.ASSERT_STATUS(row.get("expectedStatus"));
                default -> throw new
IllegalArgumentException("Unknown keyword: " + step);
            }
        }

        // Example: fetch booking id for
logging/report
        String bookingId = ke.GET_BOOKING_ID();
        System.out.println("[RUN " +
row.get("runId") + "] Booking ID: " + bookingId);
    }
}
}

```

Sample Console Output :

[RUN TC01] Booking ID: BK-958214

PASSED: ticketBookingFromCsv

=====

Default Suite

Total tests run: 1, Passes: 1, Failures: 0, Skips: 0

=====

Result :

The **Hybrid Framework** successfully automated the ticket booking workflow using **Page Objects** (encapsulated locators/actions), **Keywords** (business-level steps), and **Data-Driven** CSV inputs.

ExNo: 8 Develop a hybrid framework and associated test cases for a hotel room booking system.

Aim:

To design and implement a Hybrid Automation Framework for a hotel room booking system using Java, Selenium WebDriver, TestNG, and Excel for test data. The framework should support reusable components, maintainability, and scalability, along with detailed test reporting.

Description:

The Hybrid Framework is a combination of **Data Driven** (reading input data from Excel) and **Keyword Driven** (using action keywords for reusable methods) approaches, structured with the **Page Object Model** (POM).

This framework automates the process of:

1. Navigating to the hotel booking application.
2. Logging in with provided credentials.
3. Searching for a hotel room.
4. Booking the room.
5. Validating the booking confirmation.

Java SDET Hybrid Framework Code (Simplified Example)

1. Utility - ExcelReader.java

```
package utils;

import java.io.FileInputStream;
import org.apache.poi.ss.usermodel.*;
```



```

public class ExcelReader {
    String path;
    Workbook workbook;

    public ExcelReader(String path) {
        this.path = path;
        try {
            FileInputStream fis = new
FileInputStream(path);
            workbook = WorkbookFactory.create(fis);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public String getCellData(String sheetName, int
rowNum, int colNum) {
        Sheet sheet = workbook.getSheet(sheetName);
        Row row = sheet.getRow(rowNum);
        Cell cell = row.getCell(colNum);
        return cell.toString();
    }
}

```

2. Page Object - LoginPage.java

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class LoginPage {
    WebDriver driver;

```

```

By username = By.id("username");
By password = By.id("password");
By loginBtn = By.id("login");

public LoginPage(WebDriver driver) {
    this.driver = driver;
}

public void login(String user, String pass) {
    driver.findElement(username).sendKeys(user);
    driver.findElement(password).sendKeys(pass);
    driver.findElement(loginBtn).click();
}
}

```

3. Test Case - HotelBookingTest.java

```

package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.*;
import pages.LoginPage;
import utils.ExcelReader;

public class HotelBookingTest {
    WebDriver driver;
    ExcelReader excel;

    @BeforeClass
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }
}

```

```

excel = new ExcelReader("testdata/HotelData.xlsx");
    }

    @Test
    public void bookHotelTest() {
        driver.get("https://adactinhotelapp.com/");

        String user = excel.getCellData("Sheet1", 1, 0);
        String pass = excel.getCellData("Sheet1", 1, 1);

        LoginPage lp = new LoginPage(driver);
        lp.login(user, pass);

        // Assume booking steps here...
        String title = driver.getTitle();
        Assert.assertTrue(title.contains("Search Hotel"));
    }

    @AfterClass
    public void tearDown() {
        driver.quit();
    }
}

```

Sample Output (Console)

```
csharp
CopyEdit
[INFO] TestNG starting execution...
[INFO] Opening Chrome browser
[INFO] Navigated to Hotel Booking Application
[INFO] Logging in with username: testuser
[INFO] Hotel booking steps executed
[PASS] Booking test passed
[INFO] Browser closed successfully
=====
HotelBookingTest
Tests run: 1, Failures: 0, Skips: 0
=====
```

Result:

The hybrid framework successfully automated the hotel room booking process using a combination of Data Driven and Keyword Driven approaches with the Page Object Model.

Ex.No 9: Design and develop a hybrid framework and relevant test cases for a hospital appointment application.

Aim:

To design and implement a **Hybrid Automation Framework** for a Hospital Appointment Booking Application using **Java, Selenium, and TestNG**, enabling efficient automation through reusable modules, externalized test data, and keyword-driven execution.

Description:

- **Hybrid Framework** merges the benefits of **Data-Driven Framework** (test data from Excel/CSV) and **Keyword-Driven Framework** (actions defined in keywords).
- The framework will:
 - Read test data from Excel.
 - Execute actions based on keywords.
 - Separate locators, actions, and data for maintainability.
 - Provide reusability and easy scalability.
- **Modules Included:**
 - Browser Initialization
 - Login
 - Appointment Booking
 - Logout

BaseTest.java:

```
package hybridFramework;  
  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.chrome.ChromeDriver;
```

```

public class BaseTest {
    public static WebDriver driver;

    public static void launchBrowser(String url) {
        System.setProperty("webdriver.chrome.driver",
"drivers/chromedriver.exe");
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get(url);
    }

    public static void closeBrowser() {
        driver.quit();
    }
}

```

ExcelUtils.java :

```

package hybridFramework;

import java.io.FileInputStream;
import org.apache.poi.ss.usermodel.*;

public class ExcelUtils {
    static Workbook workbook;

    public static Object[][] getTestData(String filePath,
String sheetName) throws Exception {
        FileInputStream fis = new
FileInputStream(filePath);
        workbook = WorkbookFactory.create(fis);
    }
}

```

```

        Sheet sheet = workbook.getSheet(sheetName);
        int rowCount = sheet.getPhysicalNumberOfRows();
        int colCount = sheet.getRow(0).getLastCellNum();

        Object[][] data = new Object[rowCount -
1][colCount];
        for (int i = 1; i < rowCount; i++) {
            Row row = sheet.getRow(i);
            for (int j = 0; j < colCount; j++) {
                data[i - 1][j] =
row.getCell(j).toString();
            }
        }
        return data;
    }
}

```

KeywordLibrary.java :

```

package hybridFramework;

import org.openqa.selenium.By;

public class KeywordLibrary extends BaseTest {

    public static void enterText(String locator, String
value) {
        driver.findElement(By.xpath(locator)).sendKeys(value);
    }
}

```

```
public static void clickElement(String locator) {  
    driver.findElement(By.xpath(locator)).click();  
}
```

```
public static void waitForSeconds(int sec) throws  
InterruptedException {  
    Thread.sleep(sec * 1000);  
}  
}
```

HospitalBookingTest.java :

```
package hybridFramework;
```

```
import org.testng.annotations.*;  
import java.io.IOException;
```

```
public class HospitalBookingTest extends BaseTest {
```

```
    @BeforeClass  
    public void setUp() {  
        launchBrowser("https://example-hospital-  
booking.com");  
    }
```

```
    @Test(dataProvider = "appointmentData")  
    public void bookAppointment(String username, String  
password, String patientName, String doctorName, String  
date) throws InterruptedException {
```



```
// Login
```

```
KeywordLibrary.enterText("//input[@id='username']",  
username);
```

```
KeywordLibrary.enterText("//input[@id='password']",  
password);
```

```
KeywordLibrary.clickElement("//button[@id='loginBtn']");  
KeywordLibrary.waitForSeconds(2);
```

```
// Book Appointment
```

```
KeywordLibrary.enterText("//input[@id='patientName']",  
patientName);
```

```
KeywordLibrary.enterText("//input[@id='doctorName']",  
doctorName);
```

```
KeywordLibrary.enterText("//input[@id='appointmentDate']",  
date);
```

```
KeywordLibrary.clickElement("//button[@id='bookBtn']");  
KeywordLibrary.waitForSeconds(2);
```

```
// Logout
```

```
KeywordLibrary.clickElement("//a[@id='logoutLink']");  
}
```

```
@DataProvider
```

```
public Object[][] appointmentData() throws Exception  
{  
    return  
        ExcelUtils.getTestData("testdata/HospitalData.xlsx",  
                                "Sheet1");  
}
```

```
@AfterClass
```

```
public void tearDown() {  
    closeBrowser();  
}  
}
```

username	password	patientName	doctorName	date
admin	pass123	John Doe	Dr. Smith	2025-08-20
admin	pass123	Alice Jones	Dr. Watson	2025-08-25

Sample Output(Console log):

```
Browser launched successfully.  
Entered Username: admin  
Entered Password  
Clicked Login Button  
Entered Patient Name: John Doe  
Entered Doctor Name: Dr. Smith  
Entered Date: 2025-08-20  
Clicked Book Appointment Button  
Appointment booked successfully.  
Logged out successfully.  
Browser closed.
```

Result :

The **Hybrid Framework** successfully automated the Hospital Appointment Booking Application, allowing execution of multiple test cases with varying input data from Excel.

Ex.No: 10 Implement a hybrid framework and associated test cases for an e-commerce application.

Aim:

To design and implement a **Hybrid Automation Framework** for an e-commerce web application that supports data-driven testing, keyword-driven testing, and reusable page object components, and to validate core functionalities such as product search, adding to cart, and checkout through automated test cases.

Description:

A **Hybrid Framework** combines features of multiple automation frameworks — mainly **Data-Driven**, **Keyword-Driven**, and **Page Object Model (POM)**.

In this implementation:

- **Data-Driven:** Uses Excel for test data.
- **Keyword-Driven:** Uses keywords to represent actions in scripts.
- **POM:** Separates UI locators and methods from test logic.
- **TestNG:** Provides test structure and HTML reports.
- **Apache POI:** Reads test data from Excel.
- **Log4j:** Generates detailed logs.

BaseTest.java - Test Base Class :

```
package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;

import java.time.Duration;
```

```

public class BaseTest {
    public static WebDriver driver;

    @BeforeClass
    public void setUp() {
        System.setProperty("webdriver.chrome.driver",
"drivers/chromedriver.exe");
        driver = new ChromeDriver();

driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.manage().window().maximize();
        driver.get("https://www.demoblaze.com/");
    }

    @AfterClass
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}

```

HomePage.java - Page Object Model for Home Page :

```

package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class HomePage {
    WebDriver driver;

    By searchBox = By.id("search_input");
    By firstProduct =
By.xpath("//a[contains(text(),'Samsung galaxy s6')]");

```

```
public HomePage(WebDriver driver) {
    this.driver = driver;
}

public void selectFirstProduct() {
    driver.findElement(firstProduct).click();
}
}
```

ProductPage.java - POM for Product Page :

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class ProductPage {
    WebDriver driver;

    By addToCartBtn = By.xpath("//a[text()='Add to cart']");

    public ProductPage(WebDriver driver) {
        this.driver = driver;
    }

    public void addToCart() {
        driver.findElement(addToCartBtn).click();
    }
}
```

EcommerceTest.java - Test Case with Data-Driven Excel Input

```
package tests;

import base.BaseTest;
import pages.HomePage;
import pages.ProductPage;
import org.testng.Assert;
import org.testng.annotations.Test;

public class EcommerceTest extends BaseTest {

    @Test
    public void testAddProductToCart() throws
    InterruptedException {
        HomePage home = new HomePage(driver);
        ProductPage product = new ProductPage(driver);

        home.selectFirstProduct();
        Thread.sleep(2000); // only for demo

        product.addToCart();
        Thread.sleep(2000);

        driver.switchTo().alert().accept();
        // Accept cart alert
        driver.findElement(By.id("cartur")).click();

        String cartText =
driver.findElement(By.xpath("//td[contains(text(),'Samsung
galaxy s6')]")).getText();
        Assert.assertTrue(cartText.contains("Samsung"),
"Product not found in cart!");
    }
}
```

Sample Output (TestNG HTML Report Extract)

yaml

TestNG Test Results

Suite Name: E-Commerce Suite

Total tests run: 1, Failures: 0, Skips: 0

Test Name: testAddProductToCart

Status: PASSED

Time: 5.4 seconds

Logs:

[INFO] Launching browser...

[INFO] Selecting first product...

[INFO] Adding product to cart...

[INFO] Verifying product in cart...

Result:

The **Hybrid Framework** successfully automated the core e-commerce functionality of adding a product to the cart using a combination of **Data-Driven Testing**, **Keyword-Driven Actions**, and **Page Object Model**.

Ex.No: 11 Develop a hybrid framework and design test cases for comprehensive testing of an insurance website

Aim:

To design and implement a Hybrid Test Automation Framework for an Insurance Website that combines **data-driven**, **keyword-driven**, and **Page Object Model (POM)** approaches to efficiently automate UI and functional testing.

Description:

The hybrid framework will:

- Use **POM** for separation of page locators and actions.
- Implement **keyword-driven testing** to trigger actions based on keywords in an external file.
- Use **data-driven testing** with Excel to provide dynamic test data.
- Integrate **TestNG** for test execution, reporting, and assertions.
- Support scalability for multiple insurance modules like *Policy Search*, *Premium Calculation*, and *Claim Submission*.

Java SDET Hybrid Framework Code:

pom/InsuranceHomePage.java (Page Object)

```
package pom;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
public class InsuranceHomePage {
    WebDriver driver;

    By loginLink = By.id("loginLink");
```

```

By username = By.id("username");
By password = By.id("password");
By loginBtn = By.id("loginBtn");
By policySearchBox = By.id("policyNumber");
By searchBtn = By.id("searchPolicy");
public InsuranceHomePage(WebDriver driver) {
    this.driver = driver;
}

public void clickLogin() {
    driver.findElement(loginLink).click();
}

public void login(String user, String pass) {
    driver.findElement(username).sendKeys(user);
    driver.findElement(password).sendKeys(pass);
    driver.findElement(loginBtn).click();
}

public void searchPolicy(String policyNo) {

driver.findElement(policySearchBox).sendKeys(policyNo);
    driver.findElement(searchBtn).click();
}
}

```

utils/ExcelUtils.java (Data-driven Helper)

```

package utils;

import org.apache.poi.ss.usermodel.*;
import java.io.FileInputStream;

```

```

public class ExcelUtils {

    Workbook workbook;

    public ExcelUtils(String excelPath) throws Exception {
        FileInputStream fis = new
FileInputStream(excelPath);
        workbook = WorkbookFactory.create(fis);
    }

    public String getCellData(String sheetName, int row,
int col) {
return
workbook.getSheet(sheetName).getRow(row).getCell(col).getS
tringCellValue();
    }
}

```

utils/KeywordEngine.java (Keyword-driven Engine) :

```

package utils;

import org.openqa.selenium.WebDriver;
import pom.InsuranceHomePage;

public class KeywordEngine {
    WebDriver driver;
    InsuranceHomePage homePage;

    public KeywordEngine(WebDriver driver) {
        this.driver = driver;
        homePage = new InsuranceHomePage(driver);
    }
}

```

```

        public void executeKeyword(String keyword, String
data) {
            switch (keyword.toLowerCase()) {
                case "clicklogin":
                    homePage.clickLogin();
                    break;
                case "login":
                    String[] creds = data.split(",");
                    homePage.login(creds[0], creds[1]);
                    break;
                case "searchpolicy":
                    homePage.searchPolicy(data);
                    break;
                default:
                    System.out.println("Invalid keyword: " +
keyword);
            }
        }
    }
}

```

tests/InsuranceTest.java (TestNG Runner):

```

package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.annotations.*;
import utils.ExcelUtils;
import utils.KeywordEngine;

public class InsuranceTest {
    WebDriver driver;
    ExcelUtils excel;
    KeywordEngine keywordEngine;
}

```

```

    @BeforeClass
    public void setup() throws Exception {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.get("https://example-insurance.com");
        excel = new ExcelUtils("data/insuranceData.xlsx");
        keywordEngine = new KeywordEngine(driver);
    }

    @Test
    public void testInsuranceWorkflow() {
        for (int i = 1; i <= 3; i++) {
            String keyword =
excel.getCellData("TestSteps", i, 0);
            String data = excel.getCellData("TestSteps",
i, 1);
            keywordEngine.executeKeyword(keyword, data);
        }
    }

    @AfterClass
    public void tearDown() {
        driver.quit();
    }
}

```

Sample Output:

[TestNG] Running:
Command line suite

Clicked on Login link.
Entered username and password.
Searched for policy: POL12345
Test Completed Successfully.

=====

Default Suite

Total tests run: 1, Failures: 0, Skips: 0

=====

Result :

The Hybrid Framework successfully executed automated test cases for the Insurance Website using a combination of **Page Object Model**, **Data-driven testing**, and **Keyword-driven testing**.